Testování softwaru

Přednáška 1:

Úvod, typy testů, V-model, W-model, black-box a white-box testing, testovací scénář, proces opravy chyb

Miroslav Bureš 2016

Úvod

Proč testovat software?

- Lidé se mohou dopustit **omylu**
- Vlivem lidské chyby se do systému naimplementuje **defekt (chybu, bug)**
- Pokud se defekt aktivuje, způsobí **selhání** systému

- Defekty je potřeba najít, než se systém začne používat
- To je možné dělat s velmi různou efektivitou
- Testování, pokud má být efektivní, je poměrně silně závislé na spolupráci s ostatními částmi vývojového cyklu

Jak typicky vypadá testování software?

- Vykonáváme činnost v aplikaci podle předpisu, kde je činnost a očekávaný výsledek
- Pokud není očekávaný výsledek dosažený, nahlásíme chybu, kterou vývojáři opravují. Opravu pak přetestujeme.
- Ale to je jen úplně základní postup, testování je mnohem zajímavější

Konvence

- Terminologie v testování je relativně nejednotná, proto pro tento předmět:

- Základní teorie testování a terminologie podle **ISTQB foundation level sylabu** (viz literatura)
- Termíny z oblasti testování použité z ISTQB glossary v anglické verzi
- Místy použitá metodika **TMAP Next**, tak aby nepřekrývala ISTQB standard

- Začneme jednodušším pohledem na problematiku, postupně půjdeme do detailu

ISTQB

- International Software Testing Qualification Board
- http://www.istqb.org/

TMAP *Next*

- Test Management Approach
- http://www.tmap.net/

Historie SW testování

Globálně

- 1956 - Debugging
- **1957 - Splňuje SW specifikaci?**
- 1979 - Přímý cíl hledat chyby v SW
- 1983 - Definice metrik, vyhodnocování
- 1988 - Zaměření na prevenci

V Čechách (hrubá aproximace):

- Dřív testování průmyslových aplikací než SW v ostatních oblastech
- Větší povědomí o testingu jako samostatné disciplíně cca od roku 2000
- Systematický testing – do SW firem se začíná dostávat cca od 2004
- Nyní vnímáno jako samostatná disciplína skoro všude, ale oproti ostatním částem vývojového cyklu lehce podceňovaná

7 základních principů testování

#1 Testování ukazuje přítomnost defektů

- Testování může ukázat, že jsou defekty přítomny, ale nemůže dokázat, že v softwaru nejsou žádné defekty.
- Testování snižuje pravděpodobnost, že v softwaru zůstanou neobjevené defekty, avšak nenalezení žádného defektu stále není důkaz bezchybosti.

#2 Vyčerpávající testování je nemožné

- Testování všeho (všech kombinací vstupů a vstupních podmínek) není realizovatelné s výjimkou triviálních případů.
- Namísto vyčerpávajícího testování by měly být k určení hlavního předmětu testovacího úsilí použity analýza rizik a stanovení priorit.

#3 Včasné testování

- Testovací aktivity by měly v rámci životního cyklu vývoje softwaru nebo systému začít tak brzy, jak je to jen možné, a měly by být zaměřeny na definované cíle.

#4 Shlukování defektů

- Malé množství modulů obsahuje většinu defektů zjištěných v průběhu testování před uvolněním, nebo je zodpovědné za nejvíc provozních selhání.

7 základních principů testování

#5 Pesticidní paradox

- Jsou-li stále opakovány tytéž testy, časem stejný soubor testovacích případů nenalezne žádné nové defekty.
- K překonání tohoto „pesticidního paradoxu“ je potřeba existující testovací případy pravidelně revidovat a upravovat a je potřeba napsat nové, odlišné testy na vykonání jiných částí softwaru nebo systému pro případné odhalení dalších defektů.

#6 Testování je závislé na kontextu

- Testování je vykonáváno odlišně v různých kontextech.
- Například software kritický z pohledu bezpečnosti se testuje jiným způsobem než webové stránky elektronického obchodu.

#7 Falešná představa o neexistenci omylů

- Nalezení a opravení defektů nepomůže, pokud je vytvořený systém nepoužitelný a nesplňuje potřeby a očekávaní uživatelů.

(převzato z ISTQB FL sylabu)

Typy testů

- Typy testů se dají kategorizovat podle několika různých pohledů
- Na začátek jen pro ukázku. V detailu probereme vše postupně později.

Podle test levels:
- Vývojářské testy
- Integrační testy
- Systémové testy
- Uživatelské akceptační testy
- …

Podle znalosti vnitřní struktury:
- Black-box testing
- White-box testing
- Grey-box testing

Podle technického postupu testu nebo charakteristiky kvality systému:
- Unit testy kódu
- Funkční testy
- E2E (end-to-end) testy
- Zátěžové testy
- Penetrační testy
- Testy provozuschpnosti na platformě
- Crash testy a testy zotavení systému
- …

Podle toho, kdo a kde testuje:
- Alfa testy
- Beta testy

A kam zařadit třeba (necháme schválně na později):
- Regresní testy
- Smoke testy

Black box a white box

Black Box testing

Testovací scénáře vytváříme na základě analýzy specifikace nebo dokumentace funkcionality systému, neznáme vnitřní strukturu systému

Příklad: funkční testovací scénáře vytvořené podle případů užití

White Box testing

Testovací scénáře vytváříme na základě známé vnitřní struktury systému

(synonyma: Clear Box testing, Glass Box testing)

Příklad: statická analýza kódu

Grey Box testing

Kombinace Black Box a White Box postupu zároveň

Verifikace a validace

Podle IEEE 601:

Verifikace

Proces vyhodnocování, jestli produkt konkrétní vývojové fáze odpovídá požadavkům na tento produkt definovaným na začátku této fáze.

Validace

Vyhodnocování software v průběhu nebo na konci jeho vývoje, jestli software odpovídá zadaným požadavků.

Podle Boehma (pouze jiná formulace):

Verifikace

Vyvíjíme ten produkt správně (podle specifikace)?

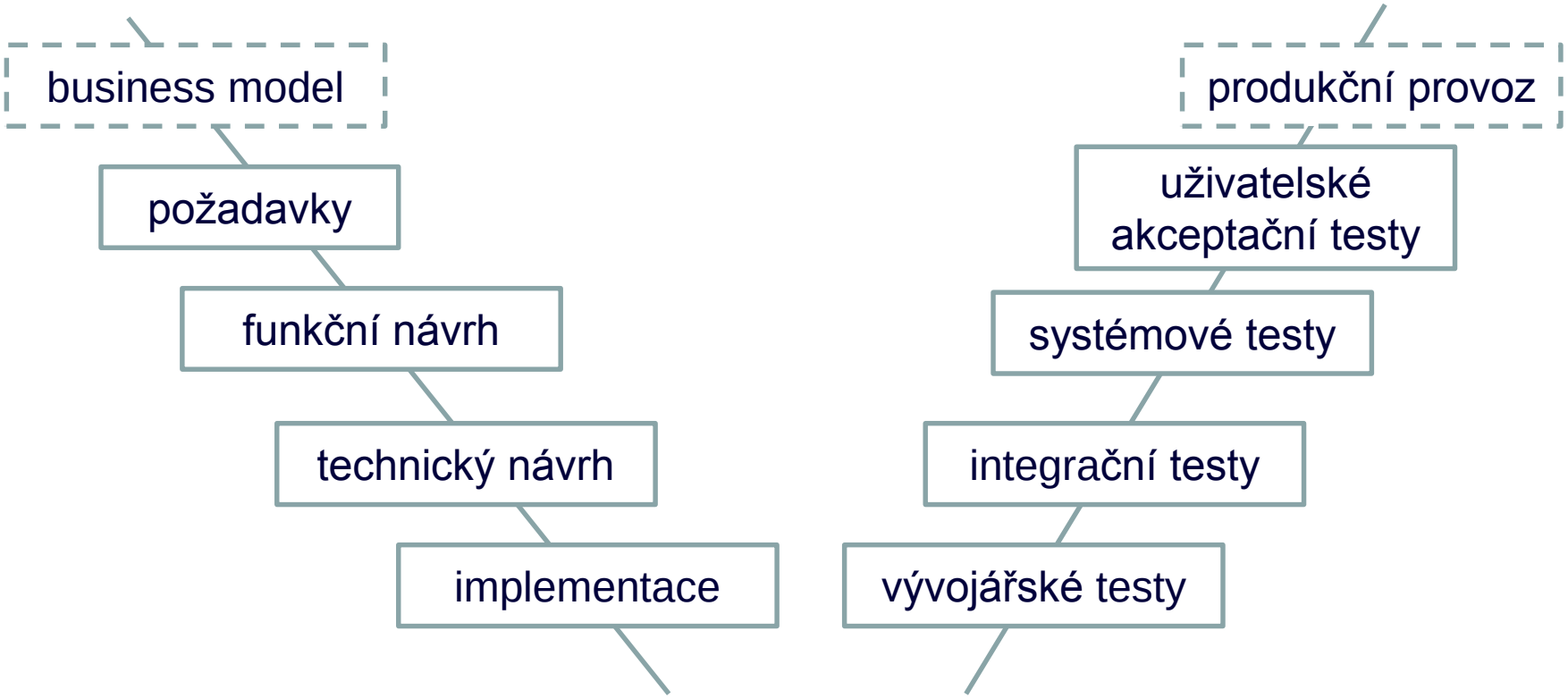
(Odpovídá produkt specifikaci, podle které se programuje?)

Validace

Vyvíjíme správný produkt (co klient požaduje)?

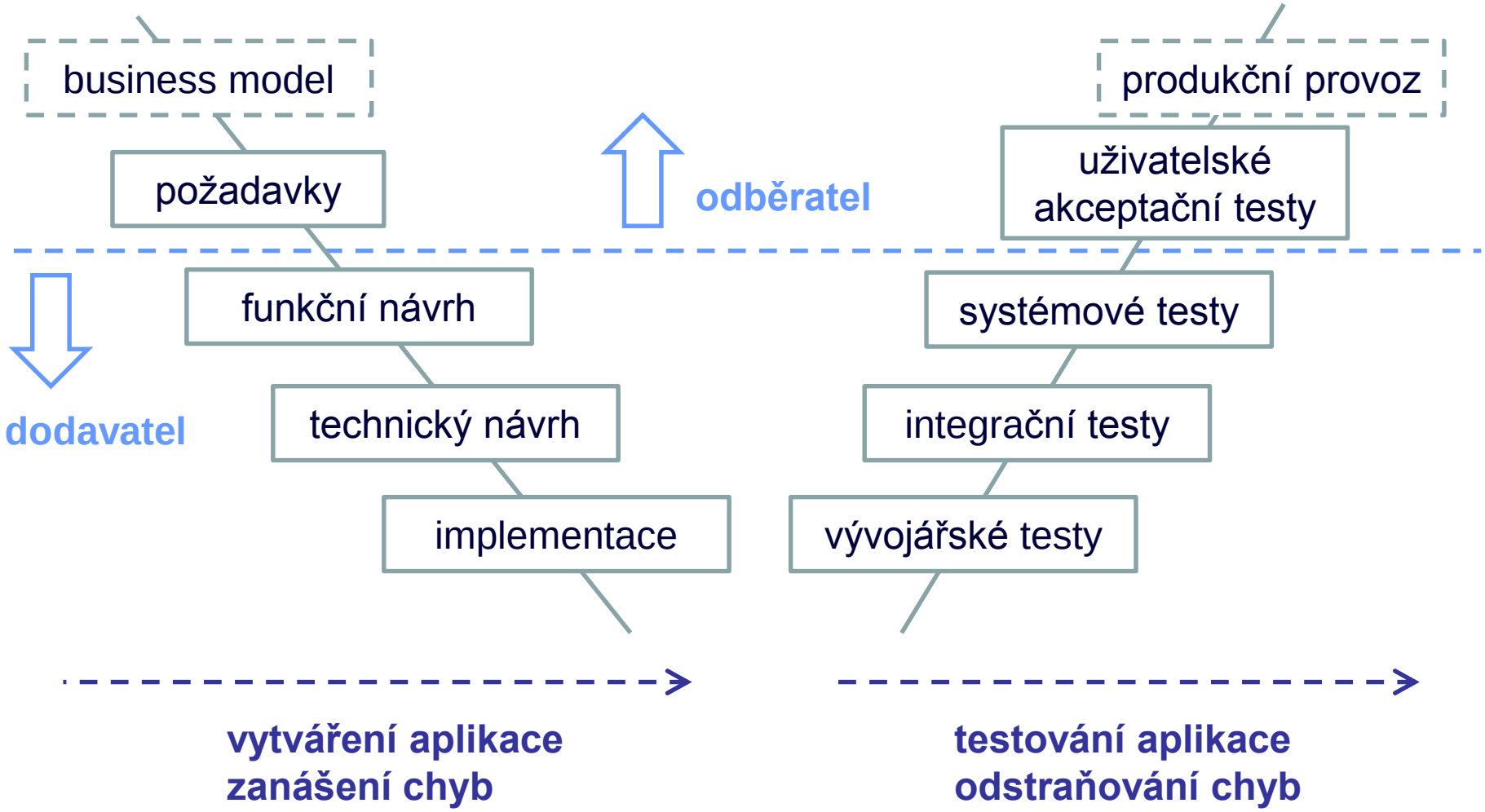
(Odpovídá produkt požadavkům klienta a jsou specifikace, podle kterých se programuje správné ve vztahu k těm požadavkům?)

V-model

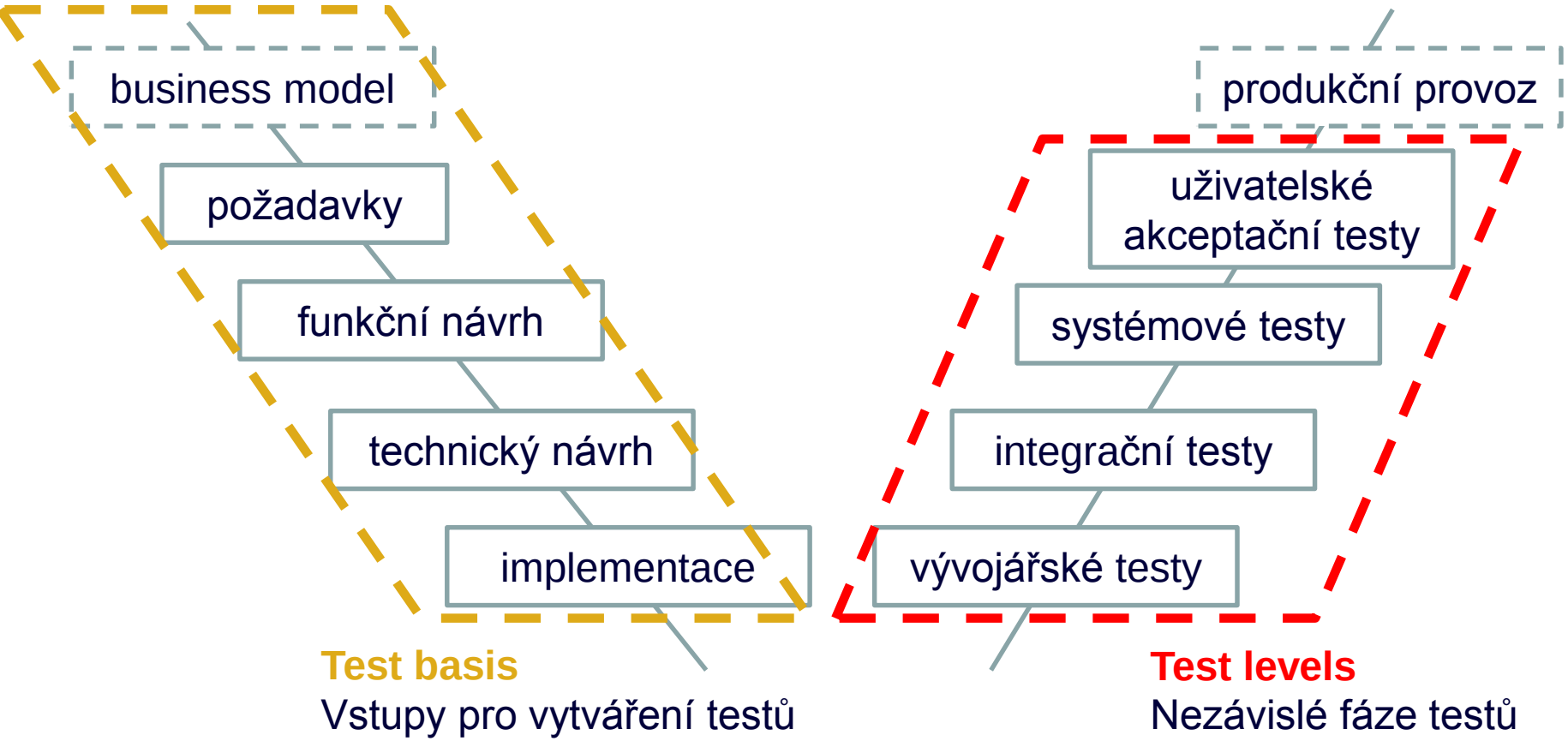


- Jednoduchá varianta podle ISTQB, kterou budeme používat
- Jsou možné modifikace pro konkrétní projekt a testovací strategii

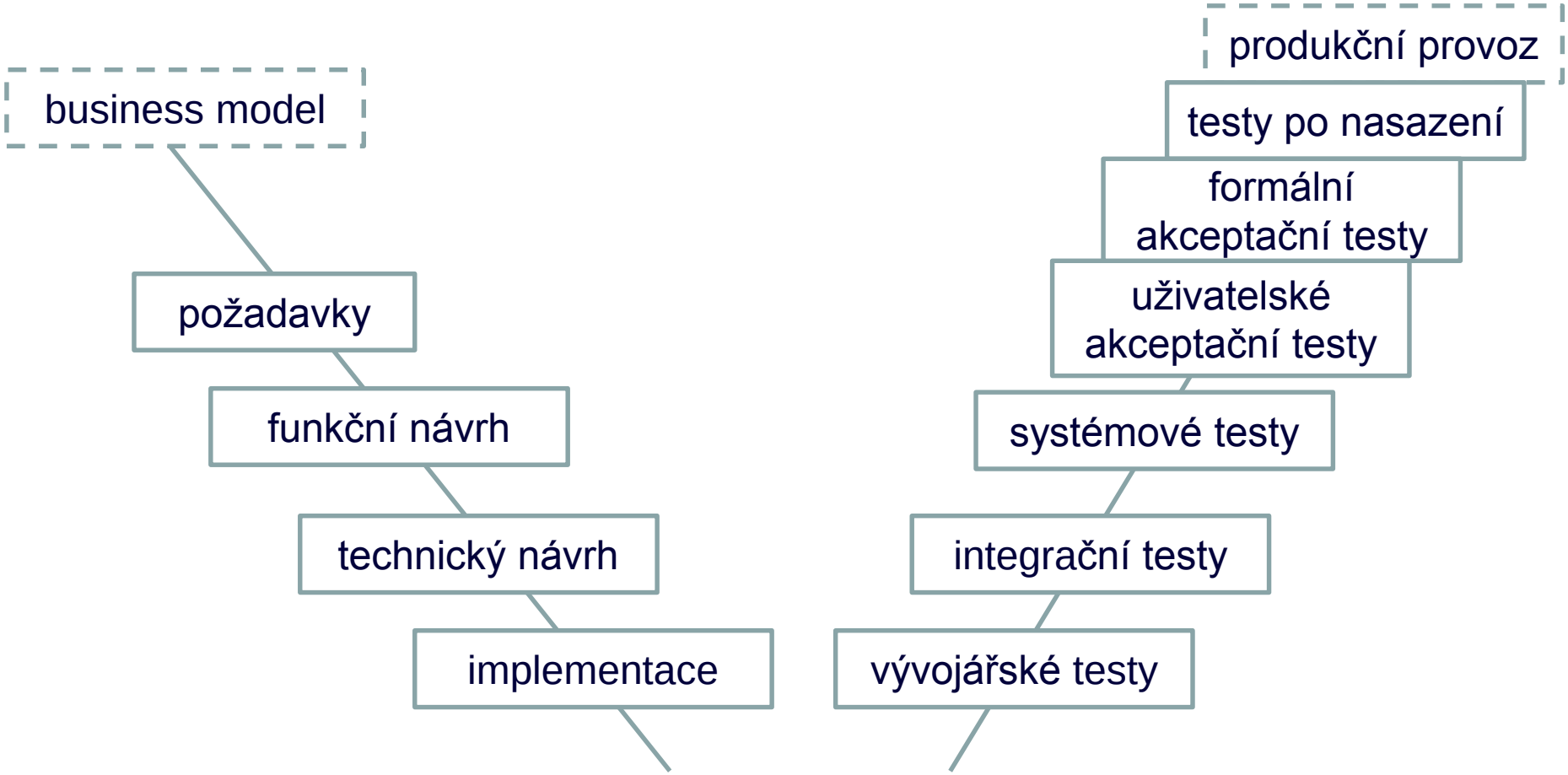
V-model


business model
požadavky
odběratel
produkční provoz
uživatelské akceptační testy
funkční návrh
systémové testy
dodavatel
technický návrh
integrační testy
implementace
vývojářské testy
vytváření aplikace
zanášení chyb
testování aplikace
odstraňování chyb

Test level, test basis



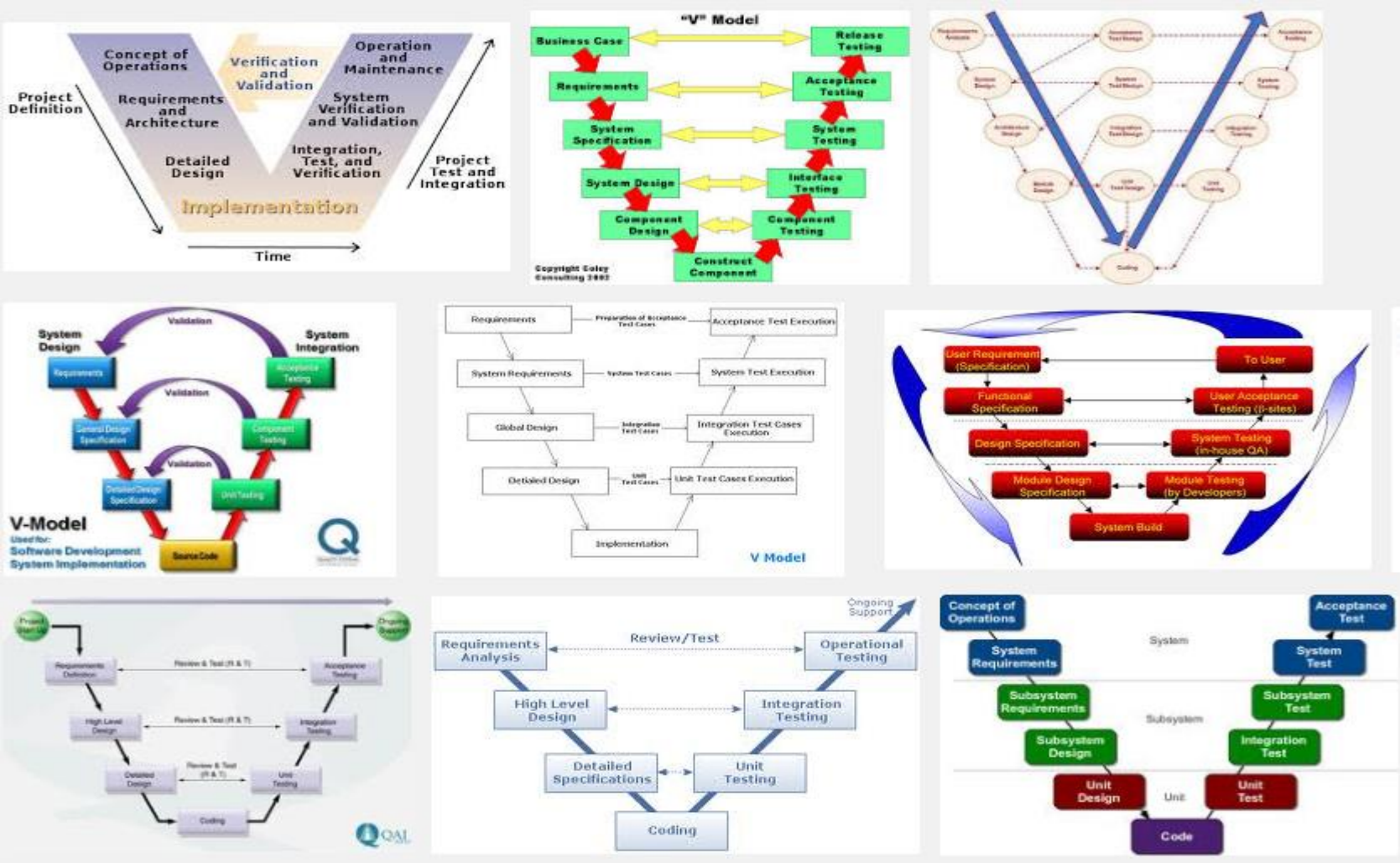
V-model



- Jak může vypadat varianta V-modelu

V-model

- **Další varianty?** Google.com, pictures, V-model
- Pricip je stále stejný



Základní termíny

Test Basis

Definice dle ISTQB: Dokumentace, na jejíž základě jsou vytvářené testovací scénáře.

Rozšířená definice: Všechny informace, na základě kterých připravujeme testy systému.

Test Level

Skupina testovacích aktivit, které jsou organizované a řízené jako jeden blok.

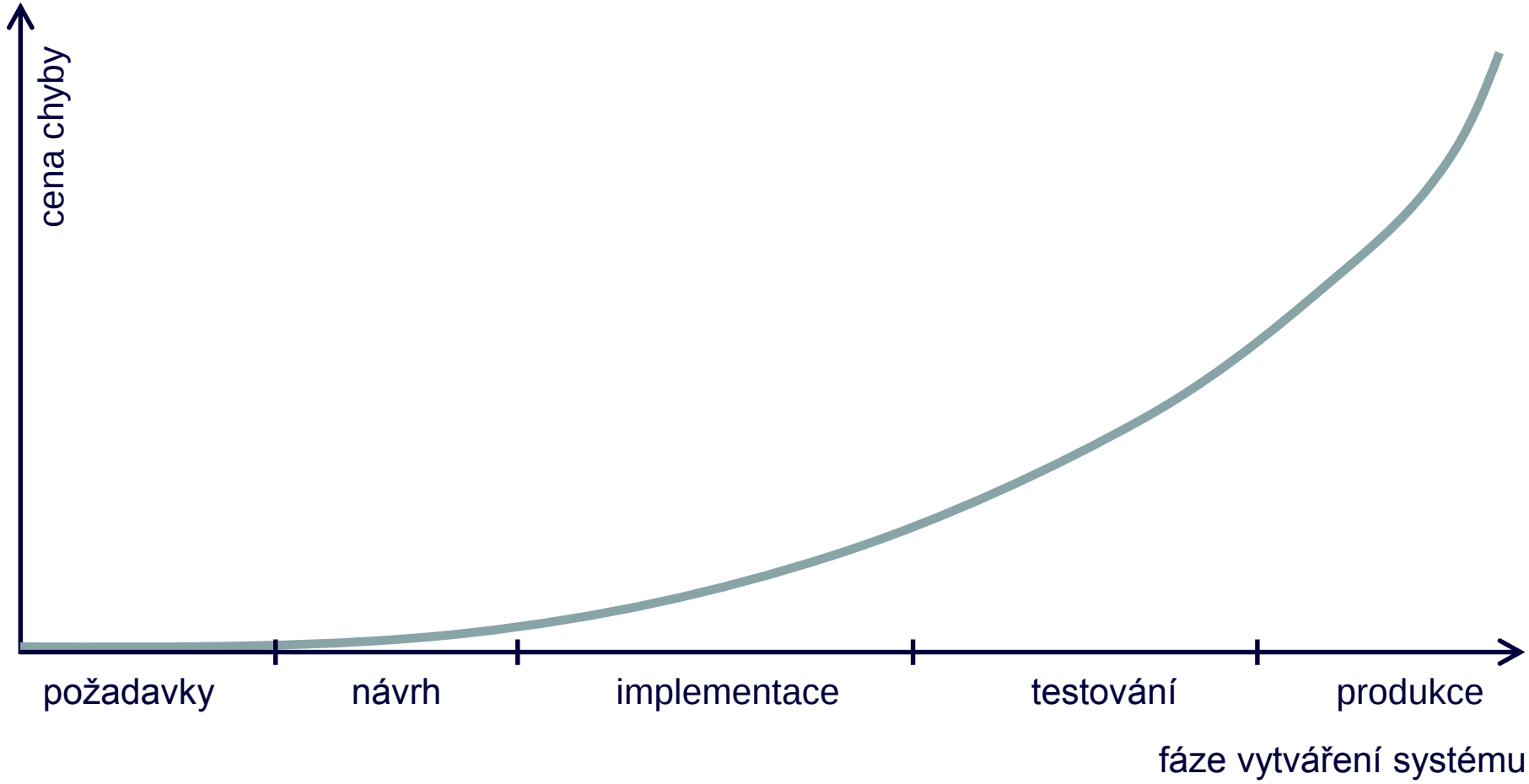
Má svůj samostatný cíl.

UAT

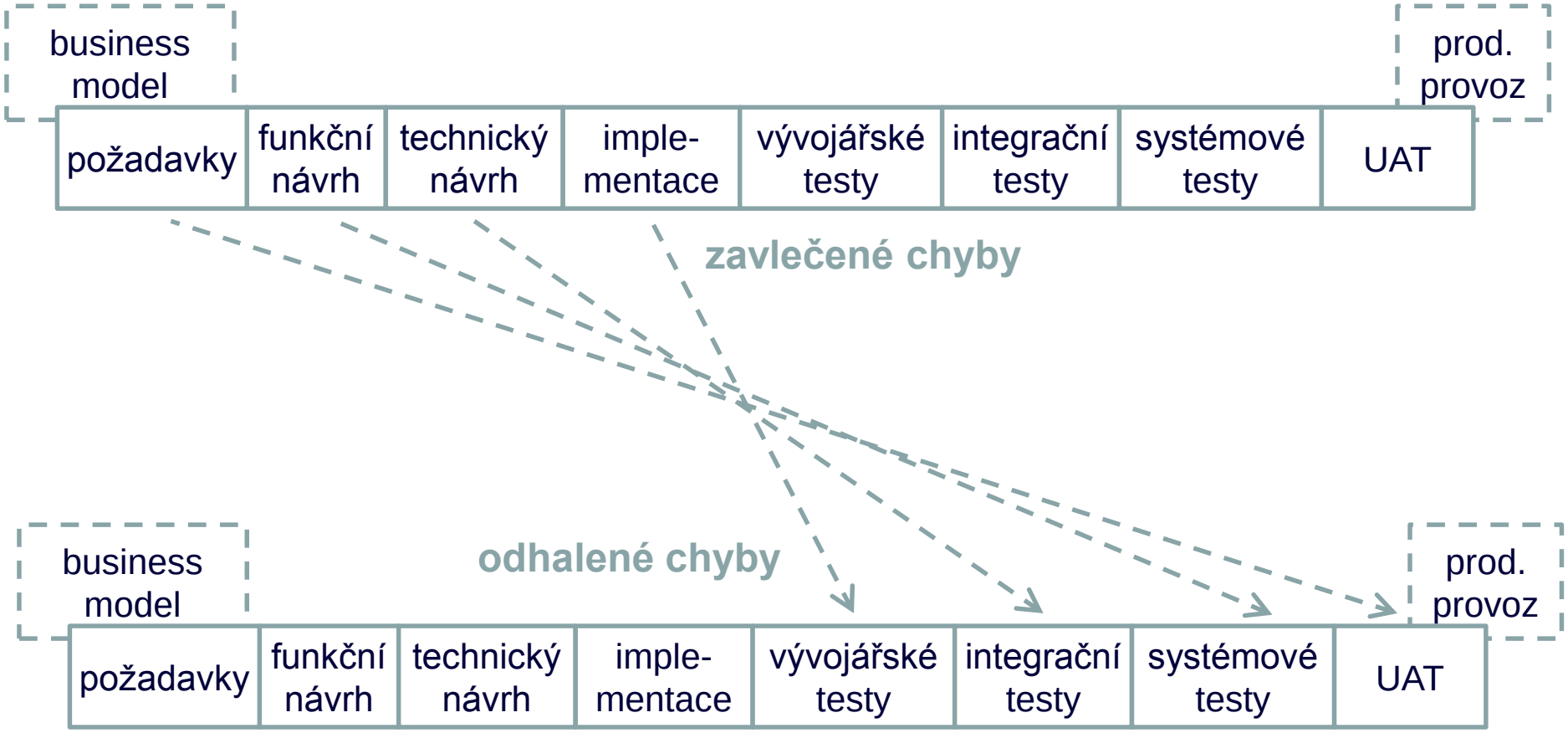
Uživatelské akceptační testy (viz výše)

Boehmův první zákon

- Chyby jsou nejčastější ve fázích požadavků a návrhu a **čím později odhalíme chybu v průběhu vývoje systému, tím je dražší**



Nevýhoda V-modelu



- V-model nakreslený v rovině
- Drahé chyby způsobené špatnými požadavky odhalujeme až ke konci testování !
- V praxi je to mírnější, ale nevhodný princip trvá

Proč je to v praxi mírnější?

W-model

- Každé fázi přípravy systému odpovídá kontrolní fáze
- V praxi levnější než předělávání a opravy na poslední chvíli, ale vyžaduje počáteční investici, proto není příliš rozšířený

Testovací scénář

- Jednoznačný předpis, jakou akci v testované aplikaci vyvolat a jaký je očekávaný výsledek

Definice dle IEEE 610:

Sada vstupních hodnot, vstupních podmínek pro vykonání scénáře, očekávaných výsledků a výstupních podmínek po provedení scénáře, vytvořený za účelem ověření konkrétní funkce, vlastnosti nebo elementu systému, například pomocí vykonání určité cesty v programu nebo ověření souladu s konkrétním požadavkem

- Scénáře připravované typicky před testem na základě test basis
- Evidence testovacích scénářů zpravidla v specializovaném nástroji, občas v praxi i v XLS
- Testovací scénáře **zastarávají** s tím, jak se aplikace mění
- I v testovacích scénářích mohou být chyby

- Terminologie zde: česky **testovací scénář** = ISTQB definice **test case**

Typy podle úrovně detailu

Dělení podle ISTQB a TMAP:

Úroveň detailu	Synonyma	Popis
High level test case (ISTQB)	Logical test case (ISTQB, TMAP) Abstract test case (ISTQB)	Testovací scénář (test case) s hodnotami pro vstupní data a očekávané výsledky specifikovanými pouze pomocí logických podmínek, nejsou přímo definována konkrétní hodnoty, které je třeba zadat.
Low level test case (ISTQB)	Physical test case (TMAP) Concrete test case (ISTQB)	Testovací scénář (test case) s konkrétními hodnotami pro vstupní data a očekávané výsledky. Logické podmínky ve specifikaci vstupních dat jsou nahrazeny konkrétními hodnotami, které odpovídají těmto podmínkám.

Jiná možnost dělení (v praxi najdete):

- Úroveň detailu popisu scénáře
- Nízká úroveň detailu – scénáře vhodné pro testery, kteří aplikaci a procesy dobře znají (např. budoucí uživatelé v UAT testech)
- Vysoká úroveň detailu - scénáře vhodné pro testery, kteří aplikaci neznají a znalost se nepředpokládá (např. studenti, kteří jako brigádu testují aplikaci)

Testovací scénář

Testovací scénář

- ID
- Název testu
- *Hloubka detailu*
- *Shrnutí testu*
- Popis testu
- Vstupní podmínky
- Testovací data
- Očekávaný výsledek
- *Autor*
- *Priorita* ★
- *Testovací prostředí* ★
- Stav přípravy
- *Pokryté požadavky*
- *Pokryté části aplikace*

Kroky testovacího scénáře

- *Název kroku testu*
- Popis kroku testu
- Očekávaný výsledek kroku testu

- Typický příklad, existují varianty
- *Méně důležité položky šedou kurzívou*
- V testovacích nástrojích často rozlišení na
  - evidenci znovupoužitelných scénářů a
  - jejich konkrétního spouštěcího plánu (tam jeden scénář může figurovat víckrát)
- Položky ★ spíš patří do spouštěcího plánu
- Testovací data je dobré parametrizovat

Spuštění testovacího scénáře

Testovací scénář

- ID
- Souhrnný výsledek testu
- Tester
- Čas prvního kroku
- Čas posledního kroku
- Navázané chyby zjištěné testovacím scénářem

Kroky testovacího scénáře

- Výsledek kroku (fail, pass, incomplete, not available, ...)
- Navázané chyby

- Výsledek scénáře a navázané chyby se typicky do testovacího scénáře agregují z jeho kroků

Pro představu

Rozsáhlý integrační projekt v bance

- Doba trvání projektu cca 4 roky
- Doba trvání testovací fáze cca 1,5 roku
- 30 IT testerů
- 60 testerů z oblasti jiných oddělení banky (budoucí uživatelé)
- 3000 scénářů pro integrační testy
- 1500 scénářů pro uživatelské testy

Pravidelné testování systému v bance, který je přístupný klientům (webová aplikace)

- 300 scénářů pro uživatelské testy

Příklad testovacího scénáře

Testovací scénář (první část)

Co byste do scénáře ještě doplnili? Co je v něm zbytečné?

Je to high-level nebo low-level test case?

Parametr	Obsah
ID testu	PAY_025
Název testu	Zadání domácí platby
Hloubka detailu	střední
Shrnutí testu	Zadání domácí platby, pozitivní průchod, datum splatnosti stejný den, částka v limitu
Popis testu	Přihlášený uživatel zadá domácí platbu s datem splatnosti stejný den, jako je den testu. Částka platby je menší než zůstatek na účtu uživatele, cílové číslo účtu existuje. Autorizace platby proběhne správně přes SMS zprávu. Platba má proběhnout a objevit se v seznamu odeslaných plateb.
Vstupní podmínky	Přihlášený uživatel, kladný zůstatek na účtu větší než 200 Kč
Testovací data	Cílový účet – 2345436/0800 Variabilní symbol – nevyplněn Konstantní symbol - nevyplněn Datum splatnosti – aktuální den Částka – 200 Kč Priorita platby - standardní
Očekávaný výsledek	Objeví se hláška „platba byla úspěšně zadána“. V seznamu zadaných plateb přibude odeslaná platba s parametry zadanými viz. testovací data.

Příklad testovacího scénáře

Testovací scénář (druhá část)

Parametr	Obsah
Autor	Jan Novák
Priorita	Střední
Testovací prostředí	TEST_4
Stav přípravy	finální

Kroky testovacího scénáře

Parametr	Obsah
Název kroku testu	Autorizace přes SMS – pokyn k zaslání SMS
Popis kroku testu	Klikni na tlačítko „zašli autorizační SMS“ a vyčkej do 5 sekund
Očekávaný výsledek kroku testu	Na testovací mobilní telefon přijde SMS zpráva s údaji o platbě shodnými s testovacími daty a autorizačním kódem pro platbu

Další termíny

Regrese

Zanášení dalších chyb do systému vlivem oprav nebo změn systému (implementace change requests)

Co tento efekt zesiluje:

- časový stres
- vývojáři neznají všechny potřebné souvislosti (ostatní komponenty, datová architektura, …)

Test coverage (testovací pokrytí)

Intuitivní popis: Jaká část software a jeho funkcí je pokryta testovacími scénáři

ISTQB definice: Stupeň do jakého je konkrétní entita nebo jednotka pro testovací pokrytí (část kódu, kombinace vstupů) testována sadou testů, vyjádřeno v procentech

- Pro exaktní změření je potřeba nadefinovat metriku pro danou situaci a typ testů

Další termíny

SLA (Service Level Agreement)

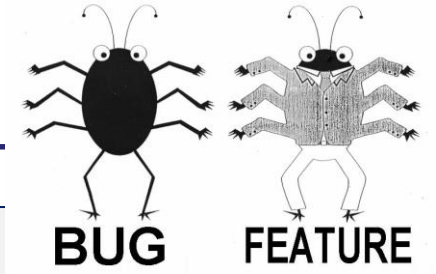
Dohoda mezi dodávající a odebírající stranou o úrovni služby. Příklady toho, co může být definováno v rámci SLA:

- Reakční doba: Např. *Práci na odstranění závažné chyby musí dodavatel začít do 4 hodin a musí být odstraněna do 12 hodin*
- Dostupnost systému: Např. *Systém musí být dostupný v režimu 24x7*

Workaround

Náhradní řešení, které má nahradit nefunkční část nebo funkcionalitu v aplikaci. Často má následující aspekty: manuální nebo částečně manuální, vyžaduje víc energie, je nepohodlný nebo rizikový. Někdy se z něj stane trvalé řešení.

Chyba (bug)



Standard	Definice bugu
ISO/IEC 9126	Defekt software nebo rozpor se specifikací
ISTQB	Nedostatek v systému nebo jeho komponentě, který brání jeho požadované funkci. Pokud se defekt aktivuje za běhu systému, může způsobit jeho pád.
TMAP *Next*	Zjištěný rozdíl mezi očekávaným nebo předpokládaným a skutečným chováním systému

Namátkově několik pohledů z praxe:

- **Priorita?** Finanční instituce a audit ČNB: „Ve vašem povinném reportu je podstatná chyba, v tiskové sestavě je 15 CZK místo 15 Kč„
- **Specifikace?** No to je pravda, že jsem vám před rokem podepsal nějaké ty diagramy. Ale my to teď potřebujeme jinak. Prostě ten software předělejte.
- **Psychologie:** Tvůrce SW předpokládá, že to co vytvořil a vidí je správné.
- **Vývojář vs. tester:** Co nám to hlásíte, to není bug, ale feature.

Priorita chyby

- Určuje přednost, s jakou budou chybu vývojáři opravovat
- Někdy se rozlišuje (typicky větší projekty):
  Severity – závažnost chyby (dopad na produkční provoz)
  Priority – přednost opravy pro vývojáře
- Bývá definováno pro konkrétní projekt (souvislost se smluvními podmínkami a SLA)

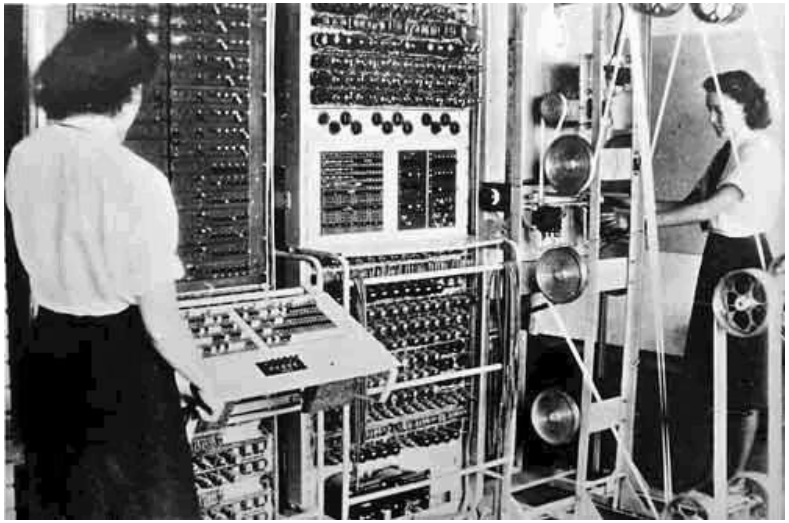
V jakých situacích to má smysl rozlišovat?

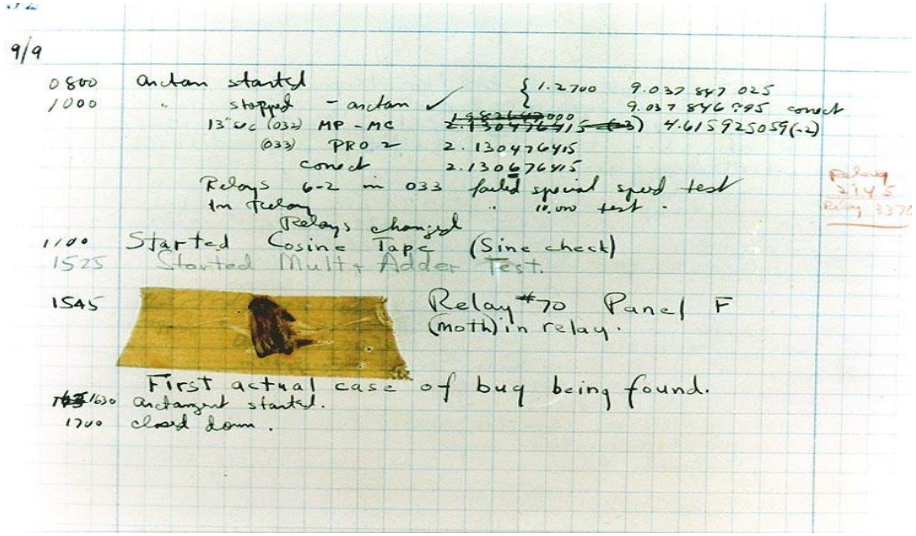
Typické stupně priority:

Priorita chyby	Popis
A - critical	Chyba blokuje podstatné procesy v aplikaci, významným způsobem poškozuje data. Neexistuje workaround.
B - medium	Chyba výrazně znepříjemňuje práci s aplikací, je možné aplikaci dočasně provozovat s touto chybou, ale workaround je obtížný a znamená velké úsilí.
C - low	Chyba se kterou je možné aplikaci provozovat, znamená akceptovatelný nekomfort, týká se nedůležitých částí nebo dat. Existuje jednoduchý workaround.

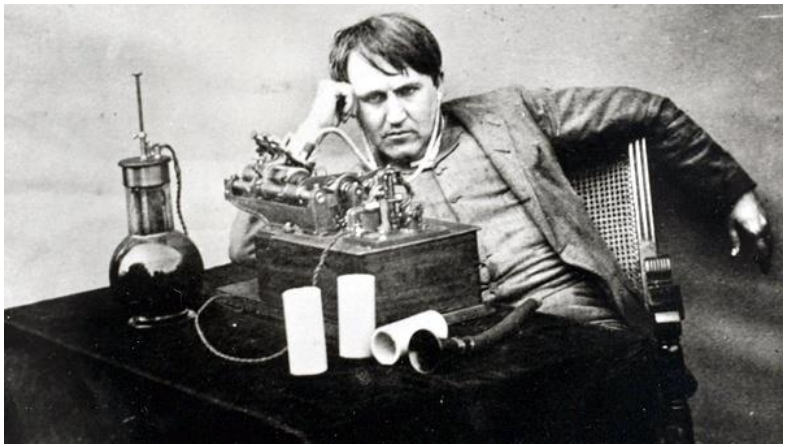
Jak vznikl termín bug?

- Klasika – 1946, můra v relé počítače MARK 2





- Termín ale pochází už z 19. století. Slovo Bug používá např. T.A.Edison.
- V češtině - slovo „moucha“ pro vadu nebo nedostatek



Pro představu

IT vývoj banky

- 200 IT vývojářů
- 100 administrátorů
- 300 business testerů
- 1 rok
- Počet evidovaných bugů a incidentů: 20 000

Vývoj jednoho systému v bance - elektronické bankovnictví

- Interní vývoj
- 1 rok
- Počet evidovaných bugů: 3 000

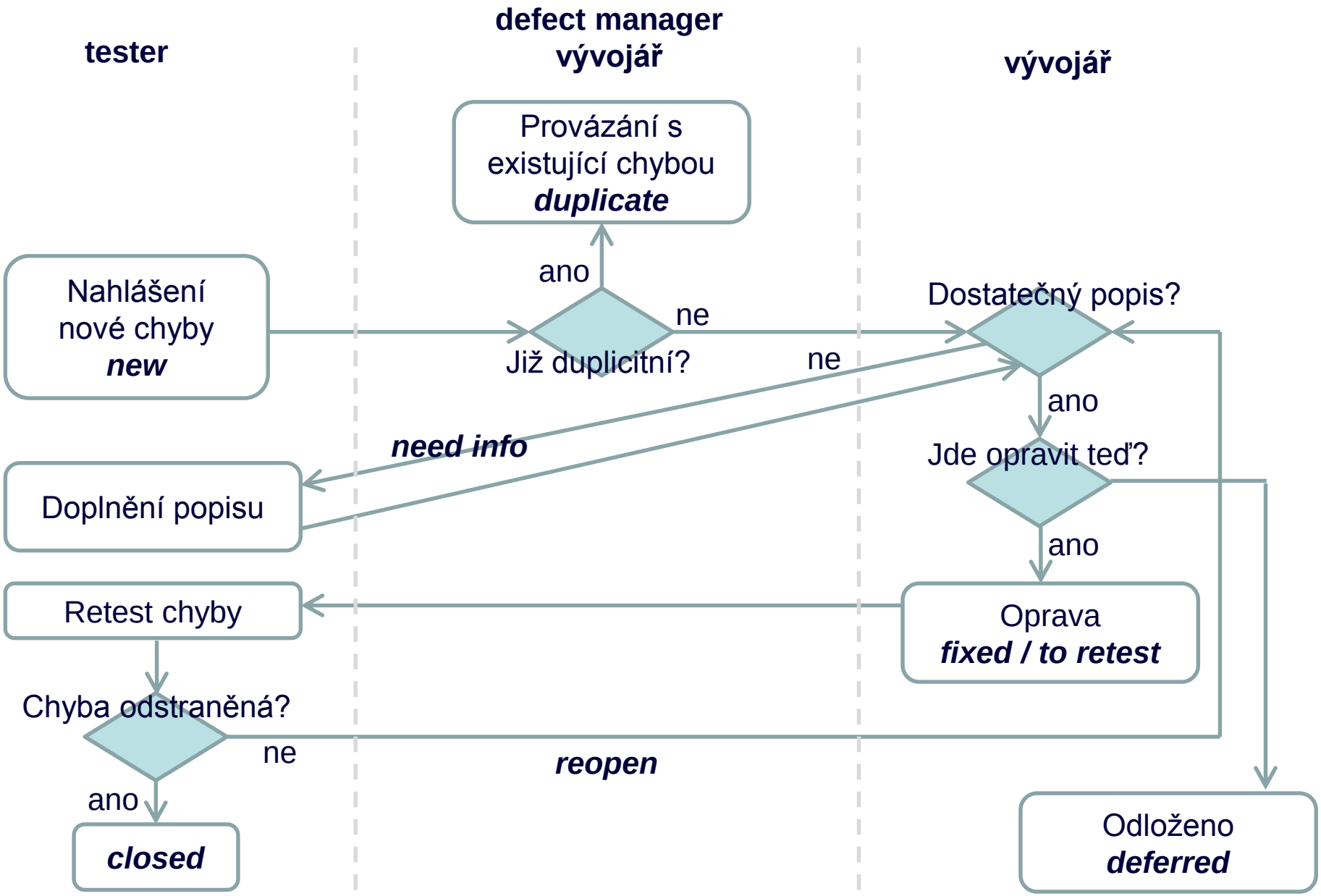
Report o chybě

- Na jeho základě vývojář znovu nasimuluje chybu, aby ji mohl lokalizovat a opravit
- Tester se nemusí pouštět do analýzy příčiny, podstatné je chybu dobře popsat

Typický report o chybě obsahuje:

- ID
- Název (shrnutí chyby)
- Odkaz na testovací scénář, který chybu odhalil
- Popis chyby
  - Kde se chyba nalezla (modul aplikace, proces, funkcionalita, interface, ...)
  - Jaké kroky vykonat, aby se chyba vyvolala
  - Nad jakými testovacími daty
  - *Další konfigurace potřebná k nasimulování chyby*
  - *Screenshot(y) z aplikace*
  - *Relevantní soubory: Chybný soubor exportu, datový soubor, atd.*
- Testovací prostředí, build, release na kterém byla chyba nalezena
- Tester, který chybu nalezl
- Čas nálezu chyby
- Stav ve workflow opravy chyb (viz další slide)

Typický cyklus opravy chyby



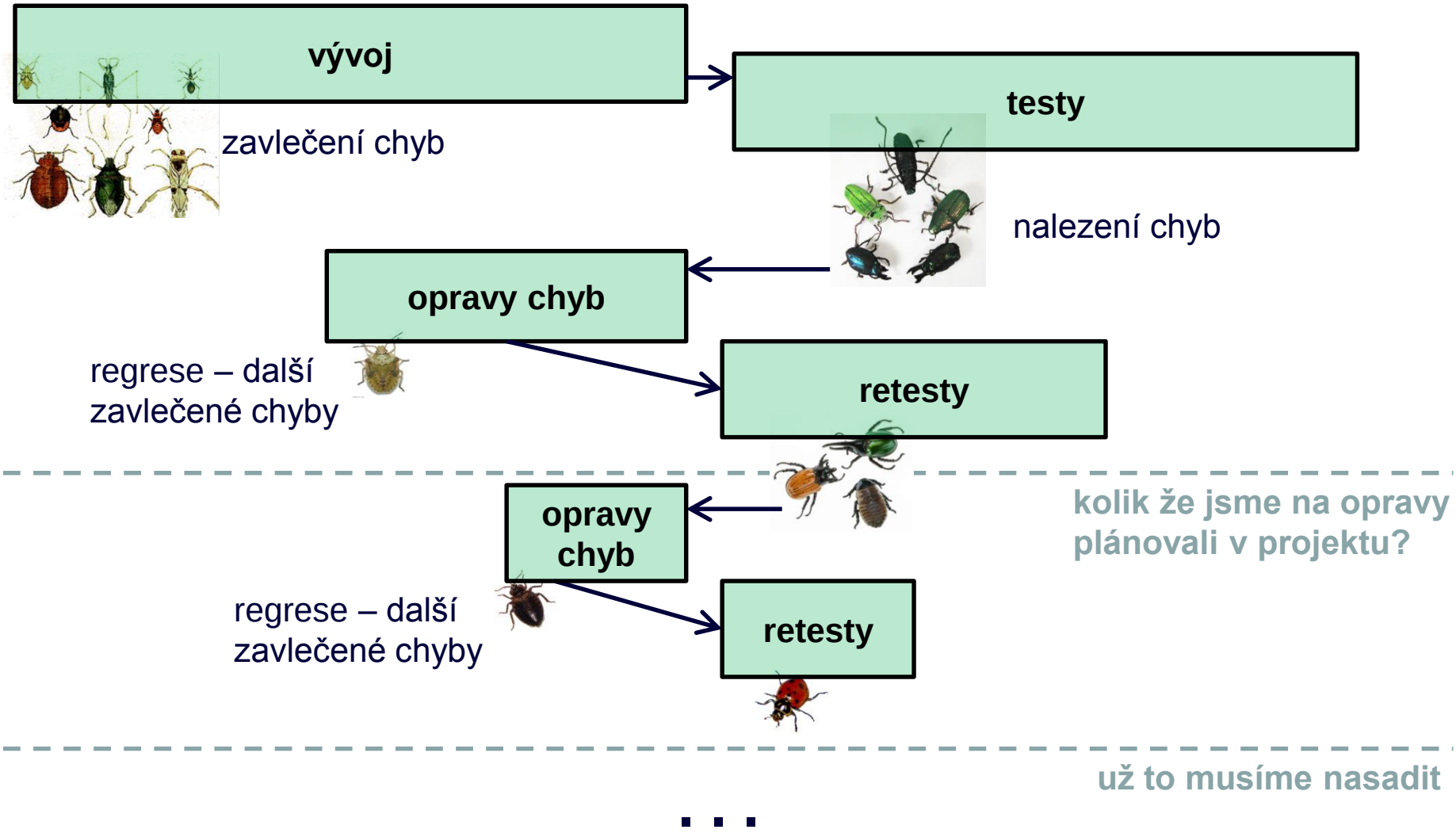
Opravy chyb na straně vývojáře

- Ideální situace je, když **vývojář chybu nemusí vracet:**
  - Pro nedostatek informací
  - Proto, že se mu nepodaří ji nasimulovat
- Nastavení SLA na opravy chyb
- „Popohánění“ ze strany testerů - je potřeba trvat na opravách a zajistit je, ale přitom pozor na:
  - Možný dopad na náladu na projektu, pokud se komunikuje nevhodně
  - Různá prioritizace z více stran devalvuje a ztrácí smysl

- Pro větší projekty má smysl zavést roli **defect managera**
  - Sleduje otevřené defekty
  - Jde přes něj prioritizace, koordinuje pořadí oprav, aby byly co nejefektivnější (nejvíc blokující chyby opravit jako první, …)

- **Komunikace chyb** ze strany testerů je důležitá
  - Komunikujte korektně, diplomaticky
  - Poukazujte na chyby, ne na konkrétní osoby
  - Působení týmu jako jeden celek – společný cíl – zprovoznit aplikaci

Opravy chyb v projektu



šířka sloupce ~ pracnost

Efektivita

Proč psát dobré testovací scénáře a popisy chyb?

dobrý scénář a popis chyby

- **tester** test
- **tester** report chyby
- **vývojář** oprava
- **tester** retest

dobrý scénář a špatný popis chyby

- **tester** test
- **tester** report chyby
- **vývojář** vrací k doplnění
- **tester** doplňuje popis
- **vývojář** oprava
- **tester** retest

špatný scénář a nerelevantní chyba

- **tester** test
- **tester** report nerelevantní chyby
- **vývojář** analýza
- **vývojář** vrací zpět

...

produktivní čas

ztráty

Děkuji za pozornost

buresm3@fel.cvut.cz